SQL ASSIGNMENT 7: ORDERS ANALYTICS & AGGREGATIONS

Subject: Database Management Systems (SQL) **Topic:** DDL, DML & Aggregate Functions **Status:** Completed

TASK 1: TABLE CREATION & DATA INSERTION

Create the `Orders` table with columns (order_id, user_name, total_amount, order_date) and insert 5 sample rows with different users and order amounts, including at least one NULL value for total_amount.

```
-- Create Orders Table CREATE TABLE Orders ( order_id INT PRIMARY KEY, user_name VARCHAR(50), total_amount DECIMAL(10, 2), order_date DATE ); -- Insert 5 Sample Records INSERT INTO Orders (order_id, user_name, total_amount, order_date) VALUES (101, 'Aarav Sharma', 1250.50, '2024-03-01'), (102, 'Priya Patel', NULL, '2024-03-02'), (103, 'Aarav Sharma', 450.00, '2024-03-03'), (104, 'Rohan Verma', 2890.75, '2024-03-04'), (105, 'Priya Patel', 799.00, '2024-03-05');
```

TASK 2: ORDER COUNT PER USER

Write a SQL query to count how many orders were placed by each user in the Orders table, displaying user_name and the number of orders as order_count.

```
SELECT user_name, COUNT(order_id) AS order_count FROM Orders GROUP BY user_name ORDER BY order_count DESC;
```

TASK 3: AVERAGE ORDER AMOUNT (IGNORING NULLS)

Write a SQL query to calculate the average total_amount of all orders in the Orders table, ensuring NULL values are ignored.

```
SELECT ROUND(AVG(total_amount), 2) AS average_order_amount FROM Orders; -- Note: SQL AVG() automatically ignores NULL values during evaluation.
```

TASK 4: FLIPKART DASHBOARD - MAX & MIN ORDER AMOUNTS

Write a SQL query to find the highest and lowest order amounts (MAX and MIN) from the Orders table, displaying both in a single result row.

```
SELECT MAX(total_amount) AS highest_order_amount, MIN(total_amount) AS lowest_order_amount FROM Orders;
```

TASK 5: TOTAL SALES CALCULATION EXCLUDING NULLS

Write a SQL query to calculate the total sales (SUM of total_amount) for all orders, filtering out NULL values using a WHERE clause.

```
SELECT SUM(total_amount) AS total_sales FROM Orders WHERE total_amount IS NOT NULL;
```